

Remove value-type invocability requirement from indirect unary callable concepts

- Abstract
- Revision history
- Motivation
- Discussion
- Wording
- References

Abstract

This is the follow up of [P2997](#).

This paper proposes removing the requirement that a callable be invocable with the value-type of an iterator in the `indirectly_unary_invocable` and related concepts. This change simplifies usage, avoids cryptic template errors, and enables support for projectors that return non-movable types.

Revision history

R0

Initial revision.

Motivation

The `indirectly_unary_invocable` currently has the following definition:

```
template<class F, class I>
concept indirectly_unary_invocable =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    invocable<F&, indirect-value-t<I>>> &&
    invocable<F&, iter_reference_t<I>> &&
    common_reference_with<
        invoke_result_t<F&, indirect-value-t<I>>>,
        invoke_result_t<F&, iter_reference_t<I>>>;
```

The red part is the *lvalue* reference to value-type of iterator in most cases, i.e., `iter_value_t<I>&`: the concept requires `F` to be invocable with both the reference type and the value-type of `I`, even if the value-type is never actually used in practice.

Unfortunately, this prevents some intuitive spellings, for example ([godbolt](#)):

```
vector<string> v;
ranges::for_each(v | views::as_rvalue, [](string&& s) { /* */ });
```

`ranges::for_each` requires `indirectly_unary_invocable`, so the lambda also needs to be able to take with `string&` even though we pass a range of `string&&`, since `string&` cannot bind to `string&&`, the constraint is not satisfied.

The error message from the compiler is even worse. From libc++ it is:

```
<source>:8:1: error: no matching function for call to object of type 'const __for_each'
 8 | std::ranges::for_each(v | std::views::as_rvalue, [](std::string&& s) { /* */ });
  | ~~~~~^~~~~~
/opt/compiler-explorer/clang-20.1.0/bin/../include/c++/v1/_algorithm/range_for_each.h:63:3: note: candidate template ignored: constraints
63 | operator()(__Range&& __range, __Func __func, __Proj __proj = {}) const {
  | ^
/opt/compiler-explorer/clang-20.1.0/bin/../include/c++/v1/_algorithm/range_for_each.h:61:13: note: because 'indirectly_unary_invocable<(l
61 | indirectly_unary_invocable<projected<iterator_t<Range>, __Proj>> __Func>
  | ^
/opt/compiler-explorer/clang-20.1.0/bin/../include/c++/v1/_iterator/concepts.h:206:60: note: because 'invocable<(lambda at <source>:8:50) &
206 | indirectly_readable<It> && copy_constructible<Fp> && invocable<Fp&, __indirect_value_t<It>> &&
  | ^
/opt/compiler-explorer/clang-20.1.0/bin/../include/c++/v1/_concepts/invocable.h:28:3: note: because 'std::invoke(std::forward<Fn>(__fn),
28 | std::invoke(std::forward<Fn>(__fn), std::forward<Args>(__args)...); // not required to be equality preserving
  | ^
/opt/compiler-explorer/clang-20.1.0/bin/../include/c++/v1/_algorithm/range_for_each.h:55:3: note: candidate function template not viable:
55 | operator()(__Iter __first, __Sent __last, __Func __func, __Proj __proj = {}) const {
  | ^
1 error generated.
```

This is cryptic and not easy to understand: the reason marked in red only shows that `invocable<lambda&, __indirect_value_t<__type>>` is false, and that's it. Unless the user actually look the implementation of `__indirect_value_t`, it is unlikely to know what going on here, which is annoying.

Moreover, this requirement for value-type invocability also blocks other reasonable use cases, for example ([godbolt](#)):

```

struct NonMovable {
    int val_;
    NonMovable(int val) : val_(val) {}
    NonMovable(NonMovable&&) = delete;
    auto operator<=>(const NonMovable&) const = default;
};

vector<int> v{1, 2, 3, 4, 5};
ranges::sort(v, {}, [](int val) { return NonMovable{val}; });

```

We are trying to project the elements into non-movable-but-comparable objects for sorting, but this is currently prohibited because `projected` has the following signature:

```

template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
using projected = /* */;

```

where `indirectly_unary_invocable` requires `common_reference_with<invoke_result_t<Proj&, indirect_value_t<I>>, invoke_result_t<Proj&, iter_reference_t<I>>>`, so we need to satisfy `common_reference_with<NonMovable, NonMovable>` since the lambda returns `NonMovable`, which in turn needs to satisfy `convertible_to<NonMovable, NonMovable>`. But that is always false because non-movable objects cannot be converted to each other.

In other words, the `common_reference_with` requirement involving `indirect_value_t<I>` in `indirectly_unary_invocable` prevents the use of projectors that return non-movable types. This is a significant limitation.

Discussion

This raises the question: why does `indirectly_unary_invocable` require value-type invocability at all?

[P2997](#) indirectly helps us trace the history of why the corresponding binary version concept, i.e., `indirect_equivalence_relation`, requires this: because we do need to construct the value-type for comparing. For example, the implementation of `ranges::unique_copy`, which relies on `indirect_equivalence_relation`, is as follows:

```

template <class _InputIter, class _OutputIter, class _BinaryPredicate,
         class _Tp>
_OutputIter __unique_copy(_InputIter __first, _InputIter __last,
                        _OutputIter __result,
                        _BinaryPredicate __binary_pred, _Tp*) {
    _Tp __value = *__first;
    *__result = __value;
    while (++__first != __last)
        if (!__binary_pred(__value, *__first)) {
            __value = *__first;
            *++__result = __value;
        }
    return ++__result;
}

```

We can see that temporary construction of the value-type is unavoidable, as `indirect_equivalence_relation` spells it out:

```

template<class F, class I1, class I2 = I1>
concept indirect_equivalence_relation =
    indirectly_readable<I1> && indirectly_readable<I2> &&
    copy_constructible<F> &&
    equivalence_relation<F&, indirect_value_t<I1>, indirect_value_t<I2>> &&
    equivalence_relation<F&, indirect_value_t<I1>, iter_reference_t<I2>> &&
    equivalence_relation<F&, iter_reference_t<I1>, indirect_value_t<I2>> &&
    equivalence_relation<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

```

It appears that the unary version simply inherits this pattern from its binary counterpart. However, this is completely unnecessary if we look at algorithms with such constrain, for example, the most typical `ranges::for_each`:

```

// for_each. Apply a function to every element of a range.
template <class _InputIter, class _Function>
_Function for_each(_InputIter __first, _InputIter __last, _Function __f) {
    for ( ; __first != __last; ++__first)
        __f(*__first);
    return __f;
}

```

we only dereference the iterator and invoke the function on the reference in each iteration, there is no value-type involved.

However, the original example is rejected because of `indirectly_unary_invocable`, which is the only reason. The code compiles successfully if we switch to `std::for_each` instead.

In the context of ranges, references are usually preferred over values. Constructing value-types is unnecessary unless required, as it can cause excessive copying. There seems to be no compelling reason to require a function to accept an iterator's lvalue value-type when accepting its reference would suffice.

Moreover, other common algorithms, such as `count_if`, etc., also do not produce value-type if we look at their implementation:

```

template <class _InputIter, class _Predicate>
typename iterator_traits<_InputIter>::difference_type
count_if(_InputIter __first, _InputIter __last, _Predicate __pred) {
    typename iterator_traits<_InputIter>::difference_type __n = 0;
    for ( ; __first != __last; ++__first)
        if (__pred(*__first))
            ++__n;
    return __n;
}

```

However, they are currently subject to this unnecessary constraint because of `indirect_unary_predicate`. Here is the summary table:

Algorithm	Unary constraint concept	Requires value-type invocation?	Explanation
<code>ranges::for_each, for_each_n</code>	<code>indirectly_unary_invocable</code>	✗	Only involve <code>invoke(f, invoke(proj, *it))</code>
<code>ranges::all_of, any_of, none_of</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::find_if, find_if_not</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::find_last_if, find_last_if_not</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::count_if</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::copy_if, replace_if, replace_copy_if, remove_if, remove_copy_if</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::is_partitioned</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::partition, stable_partition, partition_copy, partition_point</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, invoke(proj, *it))</code>
<code>ranges::filter_view, take_while_view, drop_while_view</code>	<code>indirect_unary_predicate</code>	✗	Only involve <code>invoke(pred, *it)</code>
<code>std::projected<I, Proj></code>	<code>indirectly_regular_unary_invocable</code>	✗	Only involve <code>invoke_result_t<Proj&, iter_reference_t<I>></code>
<code>std::projected_value_t<I, Proj></code>	<code>indirectly_regular_unary_invocable</code>	✓	Involve <code>invoke_result_t<Proj&, iter_value_t<I>></code>

It is worth noting that `projected_value_t` is the only one that actually requires the use of value-type of iterators by design:

```
template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
using projected_value_t =
    remove_cvref_t<invoke_result_t<Proj&, iter_value_t<I>>>;
```

This means that once we remove the value-type invocability requirement in `indirectly_regular_unary_invocable`, it is necessary to reconstrain the template parameters of `projected_value_t`:

```
template<indirectly_readable I, copy_constructible Proj>
requires invocable<Proj&, iter_value_t<I>>
using projected_value_t =
    remove_cvref_t<invoke_result_t<Proj&, iter_value_t<I>>>;
```

And this seems to be a more appropriate signature, as the main point of `indirectly_regular_unary_invocable` is to check invocability on the reference, which is irrelevant compared to the actual definition of `projected_value_t`.

In summary, the current design of indirect unary callable concepts leads to needless complexity. Relaxing them to only focus on the reference type of iterator would allow constraint algorithms to behave more sensibly without triggering cryptic errors. Furthermore, this change enables support for projectors that return non-movable objects, which is a significant improvement.

Wording

1. Bump `__cpp_lib_ranges` in 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_ranges 202406L2025XXL
// also in <algorithm>, <functional>, <iterator>, <memory>,
```

2. Modify 24.2 [\[iterator.synopsis\]](#) as indicated:

```
#include <compare>           // see [compare.syn]
#include <concepts>          // see [concepts.syn]
namespace std {
    [...]

    template<indirectly_readable I, indirectly_regular_unary_invocable<I> copy_constructible Proj>
    requires invocable<Proj&, iter_value_t<I>>
    using projected_value_t =                                // freestanding
        remove_cvref_t<invoke_result_t<Proj&, iter_value_t<I>>>;

    [...]
}
```

3. Modify 25.2 [\[indirectcallable.indirectinvocable\]](#) as indicated:

```
namespace std {
    template<class F, class I>
    concept indirectly_unary_invocable =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        invocable<F&, indirect_value_t<I>> &&
        invocable<F&, iter_reference_t<I>> &&
        common_reference_with<
            invoke_result_t<F&, indirect_value_t<I>>,>
            invoke_result_t<F&, iter_reference_t<I>>>;

    template<class F, class I>
    concept indirectly_regular_unary_invocable =
```

```

    indirectly_readable<I> &&
    copy_constructible<F> &&
    regular_invocable<F&, indirect_value_t<I>>> &&
    regular_invocable<F&, iter_reference_t<I>>> &&
    common_reference_with<
        invoke_result_t<F&, indirect_value_t<I>>>,
        invoke_result_t<F&, iter_reference_t<I>>>;

    template<class F, class I>
    concept indirect_unary_predicate =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        predicate<F&, indirect_value_t<I>>> &&
        predicate<F&, iter_reference_t<I>>>;
}

```

References

[P2997R1]

Barry Revzin. Removing the common reference requirement from the indirectly invocable concepts. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2997r1.html>